

Lecture 2 · Dynamic Programming

Bitmask · Optimisation

Bigger State Spaces · Patterns · Problems

ANCC - IIT Delhi

From Lecture 1 to Lecture 2

LECTURE 1 — FOUNDATIONS

- DP = optimal substructure + overlapping sub-problems.
- Recursion → memoization → tabulation; same $O(\text{states} \times \text{transition})$.
- Designing a DP: define state → recurrence → base → order → answer.

LECTURE 2 — BIGGER STATE SPACES

- What if the state is a SET, not an index? → Bitmask DP
- What if the table is too big or too slow? → State & space optimization

Bitmask DP

$n \leq \sim 20$

State / Space

less memory /
time

State & Space Optimization

Same recurrence, smaller footprint — shrink memory or speed up the transition without changing the DP definition. Always derive the plain DP first, then optimize.

1

Rolling array

A 2D DP that reads only the previous row keeps 1–2 rows instead of all n .

2

Implicit dimension

Derive one dimension from another instead of storing it.

3

Faster transition

Replace an $O(n)$ scan with binary search or a data structure.

Eg : LIS $O(n \log n)$

Problem Statement

Given n positive integers ($n \leq 100$, total sum $S \leq 10^5$), split them into two groups so the absolute difference of the two group sums is as small as possible. Output that minimum difference.

Idea & Observation

If one group sums to s , the other is $S-s$, giving difference $|S-2s|$. We only need WHICH sums are reachable — so the 2D subset-sum table collapses to a 1D boolean array, updated right-to-left so every item is used at most once.

STATE DEFINITION

$\text{reach}[s] = \text{true}$ if some subset sums to exactly s .

TRANSITION

for each item x : for $s = S$ down to x : $\text{reach}[s] \mid= \text{reach}[s-x]$

BASE & ANSWER

$\text{reach}[0] = \text{true}$. Answer = min over reachable s of $|S - 2s|$.

Time $O(n \cdot S)$ • Space $O(S)$

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; cin>>n;
    vector<int> a(n);
    int total=0;
    for(int &x:a){cin>>x; total+=x;}
    // reachable[s] = can some subset sum to s
    vector<char> dp(total+1,0);
    dp[0]=1;
    for(int x:a)
        for(int s=total; s>=x; --s) // BACKWARD: each item used once
            if(dp[s-x]) dp[s]=1;
    int best=INT_MAX;
    for(int s=0;s<=total;++s)
        if(dp[s]) best=min(best, abs(total-2*s));
    cout<<best<<"\n";
}
```

Problem Statement

n workers and n jobs ($n \leq 20$). $\text{cost}[i][j]$ is the cost of worker i doing job j . Assign each worker exactly one distinct job to minimize the total cost.

Idea & Observation

Assign workers in order $0, 1, 2, \dots$. The set of already-used jobs is a bitmask, and $\text{popcount}(\text{mask})$ tells you which worker comes next — so the worker index is implicit (one free dimension).

STATE DEFINITION

$\text{dp}[\text{mask}] = \text{min cost to give the jobs in mask to workers } 0.. \text{popcount}(\text{mask})-1.$

TRANSITION

$w = \text{popcount}(\text{mask});$ for unused job j : $\text{dp}[\text{mask} | 1 \ll j] = \min(\cdot, \text{dp}[\text{mask}] + \text{cost}[w][j])$

BASE & ANSWER

$\text{dp}[0] = 0.$ $\text{Answer} = \text{dp}[(1 \ll n) - 1].$

Time $O(2^n \cdot n)$ • Space $O(2^n)$

C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; cin>>n;
    vector<vector<int>> cost(n, vector<int>(n));
    for(auto&r:cost) for(int&v:r) cin>>v;
    const int FULL=(1<<n)-1;
    vector<int> dp(1<<n, INT_MAX);
    dp[0]=0;
    for(int mask=0; mask<(1<<n); ++mask){
        if(dp[mask]==INT_MAX) continue;
        int w=__builtin_popcount(mask); // next worker to assign
        if(w==n) continue;
        for(int j=0;j<n;++j) if(!(mask&(1<<j))){
            int nm=mask|(1<<j);
            dp[nm]=min(dp[nm], dp[mask]+cost[w][j]);
        }
    }
    cout<<dp[FULL]<<"\n";
}

```

Problem Statement

Given n items, each with a weight w_i and value v_i , and a knapsack of capacity W (all integers), select a subset of items to maximise total value without the total weight exceeding W — using only $O(W)$ memory.

Idea & Observation

The classic 2-D table $dp[i][j]$ only reads row $i-1$, so a single 1-D array of length $W+1$ suffices.

Iterating capacities from *high* $\rightarrow$ *low* within each item guarantees each item is picked at most once (0/1 knapsack). Swap to unbounded by iterating *low* $\rightarrow$ *high*.

STATE DEFINITION

$dp[j]$ = max value achievable
with exactly capacity j .

TRANSITION

```
for each item i (weight w, value v):
    for j = W downto w:
        dp[j] = max(dp[j],
                    dp[j - w] + v)
```

BASE & ANSWER

$dp[0..W] = 0$. Answer = $dp[W]$.

Time $O(n \cdot W)$ • Space $O(W)$

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n, W;
    cin >> n >> W;

    vector<int> w(n), v(n);
    for(int i=0; i<n; ++i)
        cin >> w[i] >> v[i];

    vector<int> dp(W+1, 0);           // dp[j] = best value at capacity j

    for(int i=0; i<n; ++i)           // iterate over items
        for(int j=W; j>=w[i]; --j)  // high→low: each item used once
            dp[j] = max(dp[j], dp[j-w[i]] + v[i]);

    cout << dp[W] << "\n";         // maximum value achievable
}

```

Problem Statement

n cities ($n \leq 18$) with a distance matrix d . Find the shortest closed tour that starts at city 0, visits every city exactly once, and returns to 0.

Idea & Observation

A subproblem is fully described by which cities are visited AND where you currently stand — the current city must be a second dimension because the remaining cost depends on it.

STATE DEFINITION

$dp[mask][last]$ = shortest path from 0 covering exactly $mask$, ending at $last$.

TRANSITION

for $nxt \notin mask$: $dp[mask | 1 \ll nxt][nxt] = \min(\cdot, dp[mask][last] + d[last][nxt])$

BASE & ANSWER

$dp[\{0\}][0] = 0$. Answer = min over last of $dp[FULL][last] + d[last][0]$.

Time $O(2^n \cdot n^2)$ · Space $O(2^n \cdot n)$

C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; cin>>n;
    vector<vector<long long>> d(n, vector<long long>(n));
    for(auto&r:d) for(long long&v:r) cin>>v;
    const long long INF=1e18;
    vector<vector<long long>> dp(1<<n, vector<long long>(n, INF));
    dp[1][0]=0; // start at city 0
    for(int mask=1; mask<(1<<n); ++mask){
        if(!(mask&1)) continue; // every path must contain city 0
        for(int last=0; last<n; ++last){
            if(dp[mask][last]==INF || !(mask&(1<<last))) continue;
            for(int nxt=0; nxt<n; ++nxt) if(!(mask&(1<<nxt))){
                int nm=mask|(1<<nxt);
                dp[nm][nxt]=min(dp[nm][nxt], dp[mask][last]+d[last][nxt]);
            }
        }
    }
    long long ans=INF, FULL=(1<<n)-1;
    for(int last=1; last<n; ++last)
        ans=min(ans, dp[FULL][last]+d[last][0]);
    cout<<ans<<"\n";
}

```

Problem Statement

An $n \times m$ hall ($m \leq 10$); '.' is a usable seat, '#' is broken. A seated student can copy from a neighbour immediately to the left, right, upper-left, or upper-right. Seat the maximum number of students with no possible copying.

Idea & Observation

Decide one row at a time. A row's occupancy is a bitmask; only the PREVIOUS row constrains the current one (the two diagonals). So carry the previous row's mask as state .

STATE DEFINITION

$dp[r][mask]$ = max students in rows $0..r$ with row 'r' seated as mask.

TRANSITION

mask valid: no broken seat, no two adjacent bits.
Compatible with p: $(mask \ll 1) \& p == 0$ and $(mask \gg 1) \& p == 0$.

BASE & ANSWER

empty profile before row 0. Answer = max over masks of $dp[n-1][mask]$.

Time $O(n \cdot 4^m)$ · Space $O(2^m)$

C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n,m; cin>>n>>m;
    vector<int> seat(n,0);           // seat[r] bit j = 1 if (r,j) is broken
    for(int r=0;r<n;++r){
        string s; cin>>s;
        for(int j=0;j<m;++j) if(s[j]=='#') seat[r]|=1<<j;
    }
    int N=1<<m;
    vector<int> prev(N,-1); prev[0]=0; // before any row: only empty profile
    for(int r=0;r<n;++r){
        vector<int> cur(N,-1);
        for(int mask=0;mask<N;++mask){
            if(mask & seat[r]) continue;           // sits on a broken seat
            if(mask & (mask<<1)) continue;         // two horizontally adjacent
            for(int p=0;p<N;++p){
                if(prev[p]<0) continue;
                if((mask<<1)&p) continue;           // upper-left conflict
                if((mask>>1)&p) continue;           // upper-right conflict
                cur[mask]=max(cur[mask], prev[p]+__builtin_popcount(mask));
            }
        }
        prev.swap(cur);
    }
    cout<<*max_element(prev.begin(),prev.end())<<"\n";
}

```

Problem Statement

n people ($n \leq 16$) and a list of incompatible pairs. Partition everyone into the minimum number of groups so that no group contains an incompatible pair.

Idea & Observation

Mark every subset that is a valid group (no internal conflict). Then it is a min set-partition: cover all people with the fewest valid subsets. Enumerate submasks (3^n total) and force the lowest set bit into the chosen subset so each partition is counted once.

STATE DEFINITION

`good[mask]` = mask has no internal conflict. `dp[mask]` = fewest valid groups covering exactly mask.

TRANSITION

`dp[mask]` = min over `sub  $\subseteq$  mask`, `sub  $\ni$  lowbit(mask)`, `good[sub]` of `dp[mask\sub] + 1`

BASE & ANSWER

`dp[0] = 0`. Answer = `dp[(1<<n)-1]`.

Time $O(3^n)$ submasks + $O(2^n \cdot n)$ precompute • Space $O(2^n)$

C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n,m; cin>>n>>m;           // m = number of incompatible pairs
    vector<int> bad(n,0);          // bad[i] = bitmask of i's enemies
    for(int e=0;e<m;++e){int u,v;cin>>u>>v; bad[u]|=1<<v; bad[v]|=1<<u;}
    int N=1<<n;
    vector<char> good(N,0);
    for(int mask=0;mask<N;++mask){
        bool ok=true;
        for(int i=0;i<n&&ok;++i) if((mask>>i&1) && (bad[i]&mask)) ok=false;
        good[mask]=ok;
    }
    vector<int> dp(N, INT_MAX);
    dp[0]=0;
    for(int mask=1;mask<N;++mask){
        int low=mask&(-mask);      // lowest set bit must be grouped now
        // enumerate submasks of mask that contain `low`
        for(int sub=mask; sub; sub=(sub-1)&mask){
            if(!(sub&low)) continue;
            if(good[sub] && dp[mask^sub]!=INT_MAX)
                dp[mask]=min(dp[mask], dp[mask^sub]+1);
        }
    }
    cout<<dp[N-1]<<"\n";
}

```

Problem Statement

Given an array of n integers ($n \leq 10^6$), find the length of its longest strictly increasing subsequence. Lecture 1 solved this in $O(n^2)$; now do it in $O(n \log n)$.

Idea & Observation

Keep $\text{tails}[k]$ = the smallest possible tail among all increasing subsequences of length $k+1$. tails stays sorted, so binary-search where x fits: replace the first entry $\geq x$ (tighten) or append (extend). The transition drops from $O(n)$ to $O(\log n)$.

STATE DEFINITION

`tails[]` sorted; its length is the current LIS length.

TRANSITION

```
it = lower_bound(tails, x);  it==end ? push_back(x) : *it  
= x.
```

BASE & ANSWER

`tails` empty. `Answer = tails.size()`.

Time $O(n \log n)$ · Space $O(n)$

C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; cin>>n;
    vector<int> a(n);
    for(int &x:a) cin>>x;
    vector<int> tails;           // strictly increasing
    for(int x:a){
        auto it=lower_bound(tails.begin(),tails.end(),x); // first >= x
        if(it==tails.end()) tails.push_back(x);           // extend
        else *it=x;
    }
    cout<<tails.size()<<"\n";
}
```

Problem Statement

n item types (capacity $W \leq 10^5$). Type i has weight w , value v , and a quantity cap c (take $0..c$ copies). Maximize total value within capacity W .

Idea & Observation

Splitting a type into c unit copies costs $O(\Sigma c)$. Instead split c into powers of two $1, 2, 4, \dots$ plus a remainder: $O(\log c)$ virtual items whose subsets compose ANY count $0..c$. Then run plain 0/1 knapsack with the 1D right-to-left trick.

STATE DEFINITION

$dp[cap]$ = best value using a prefix of the virtual items within capacity cap .

TRANSITION

for each virtual (W', V') : for $cap = W$ down to W' :
 $dp[cap] = \max(\cdot, dp[cap - W'] + V')$

BASE & ANSWER

$dp[0..W] = 0$. Answer = $dp[W]$.

Time $O(W \cdot \Sigma \log c_i)$ • Space $O(W)$

C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;
int main(){
    int n; long long W; cin>>n>>W;
    vector<long long> wt, val;           // virtual 0/1 items
    for(int i=0;i<n;++i){
        long long w,v,c; cin>>w>>v>>c;
        for(long long k=1; c>0; k<=1){  // 1,2,4,... then remainder
            long long take=min(k,c);
            wt.push_back(w*take);
            val.push_back(v*take);
            c-=take;
        }
    }
    vector<long long> dp(W+1,0);
    for(size_t i=0;i<wt.size();++i)
        for(long long cap=W; cap>=wt[i]; --cap) // BACKWARD: each virtual item once
            dp[cap]=max(dp[cap], dp[cap-wt[i]]+val[i]);
    cout<<dp[W]<<"\n";
}

```